Waypoint Navigation System using Android App: Target_Setter

by

CHHENG Lydiya

Date: 20 February 2026

Contents

- Background
- Objective(s)
- Scope of work
- Approach(es)
- Result & Discussion
- Conclusion & Next Steps

Background

- Semi-autonomous waypoint navigation system using custom Android app
- The system has 2 packages: Target_Setter (Android app) and Robot’s control package
- Target_Setter is developed to enable navigation without using gamepad as controller

Objectives

- Develop and implement waypoint navigation system including edit and return functionalities
- Test and evaluate system performance and accuracy
- Implement Android interface for semi-autonomous navigation

Scope of work

This project covers:

- Android application development (Target_Setter)
- Communication system
- Robotic software development (ROS2)
- System integration and testing

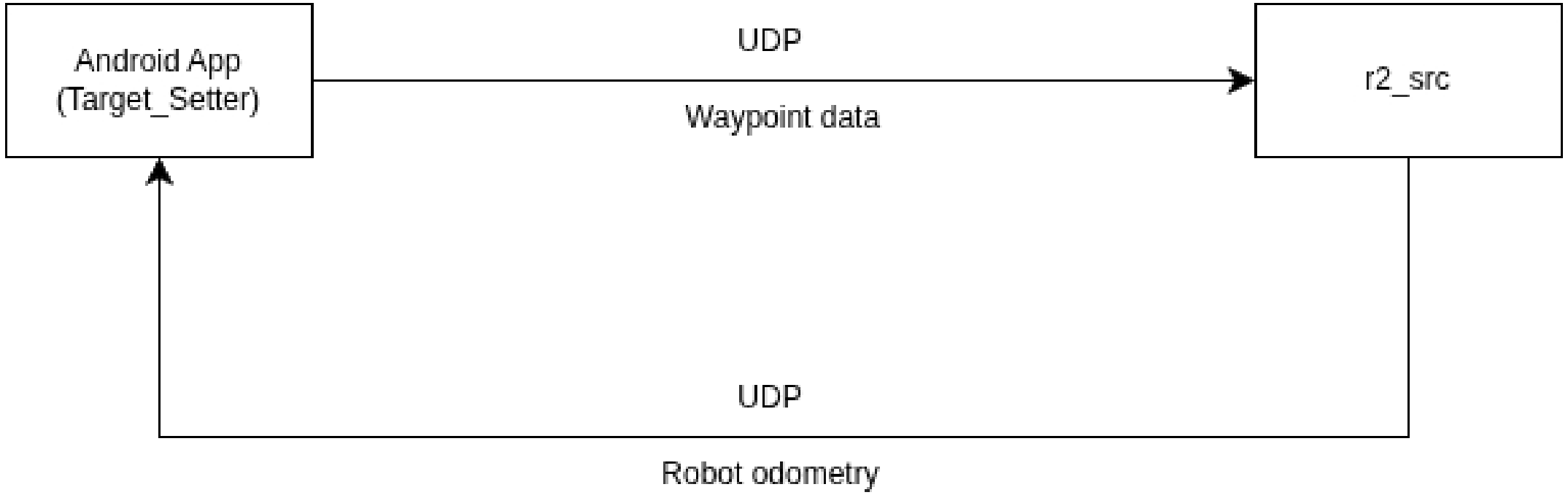
Scope of work

This project does not include:

- Full autonomous robot navigation
- Sensor fusion beyond basic odometry and IMU integration
- Algorithm optimization
- Low-level communication protocol
- Hardware-level motor driver design
- Network security mechanism

Approach

System Architecture

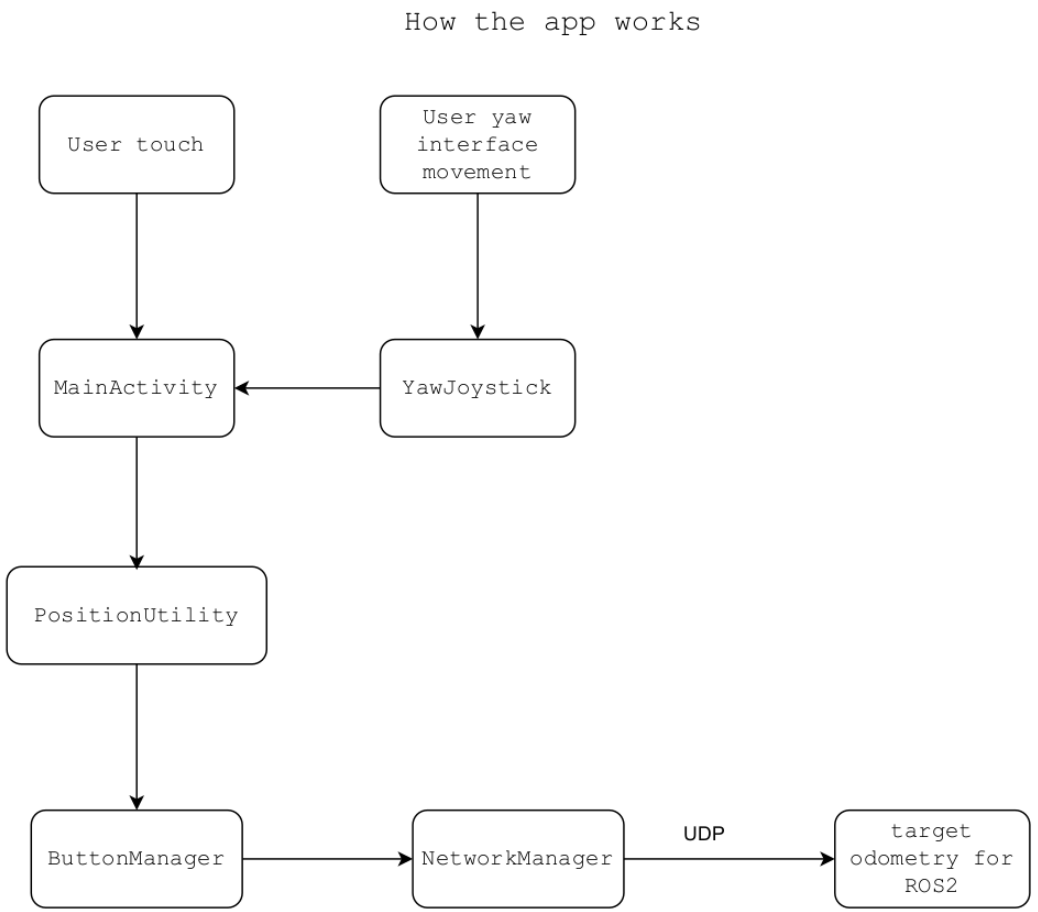


This is the high-level architecture of the system

Approach: version 0.1

System Architecture: Android app

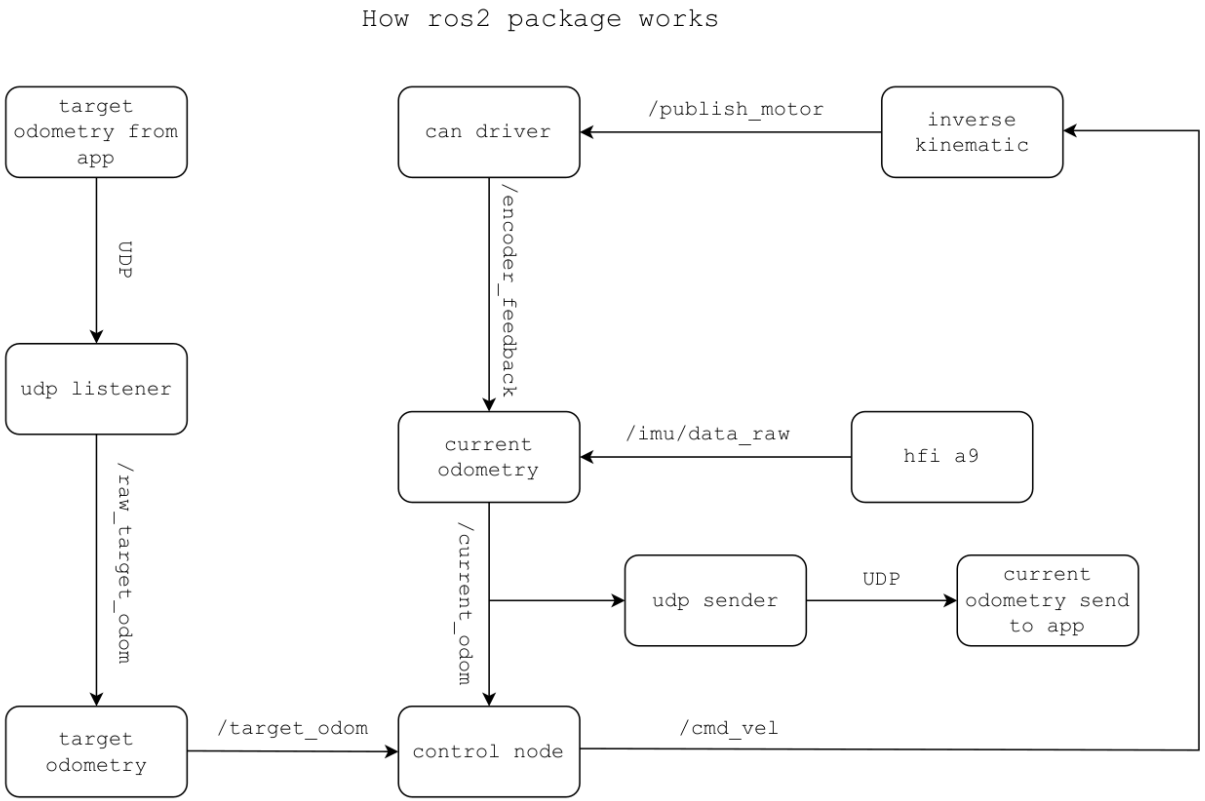
- Works based on touch input
- Implemented virtual joystick for yaw control
- Used MVC architectural pattern
- Communicated with ROS2 via UDP
- Android app handled all gamefield scaling and waypoints calculation



Approach: version 0.1

System Architecture: ROS2 package

- Takes inputs from Android app
- Implemented PD controller
- Used ROS2 framework to operate
- Communicated with app via UDP
- Handled all control logic, feedback, and navigation



Approach: version 0.1

Android app design

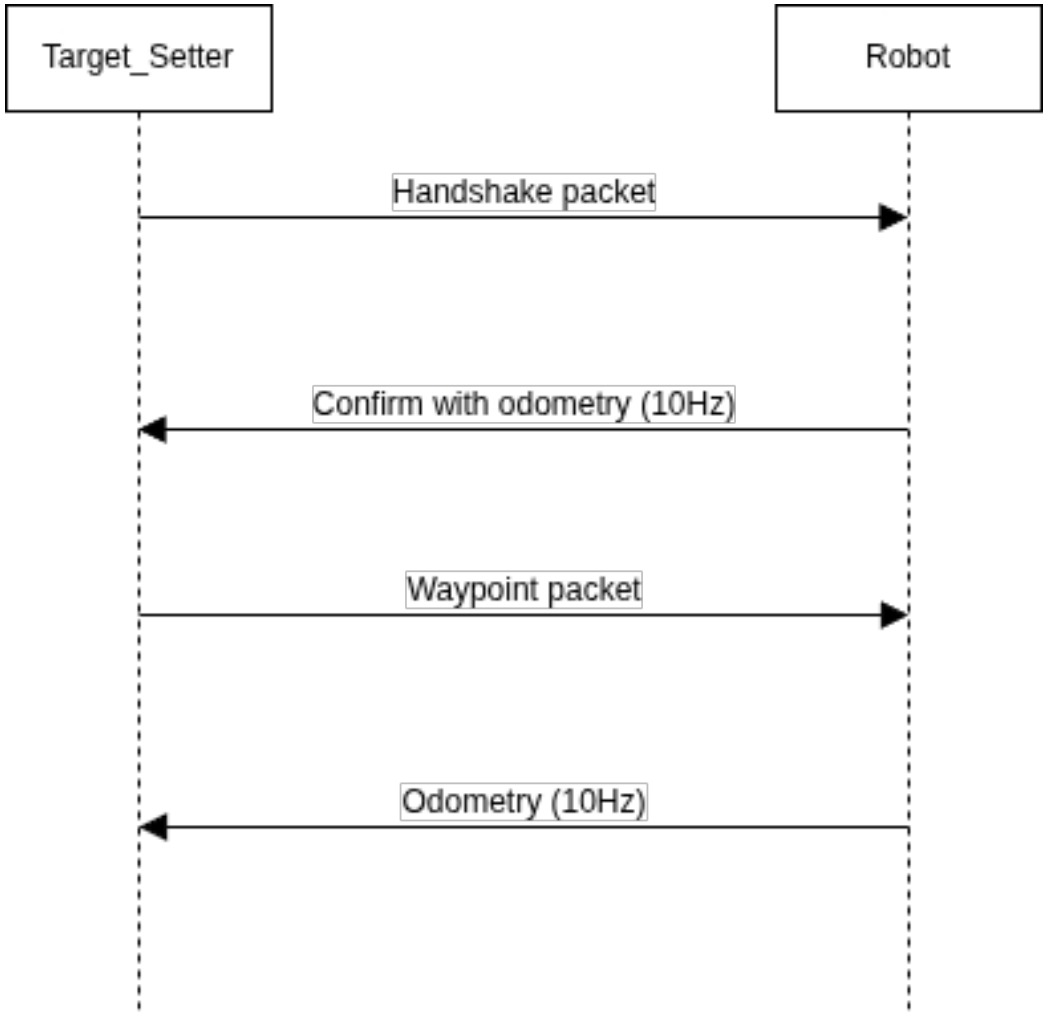
- MVC architecture
- Percentage-based coordinate storage
- ConstraintLayout for scalability

Initial version didn’t include edit and return functionalities, just basic waypoints input

Approach: version 0.1

Communication system

- UDP-based communication protocol (low latency)
- JSON array UDP packet
- Handshake mechanism



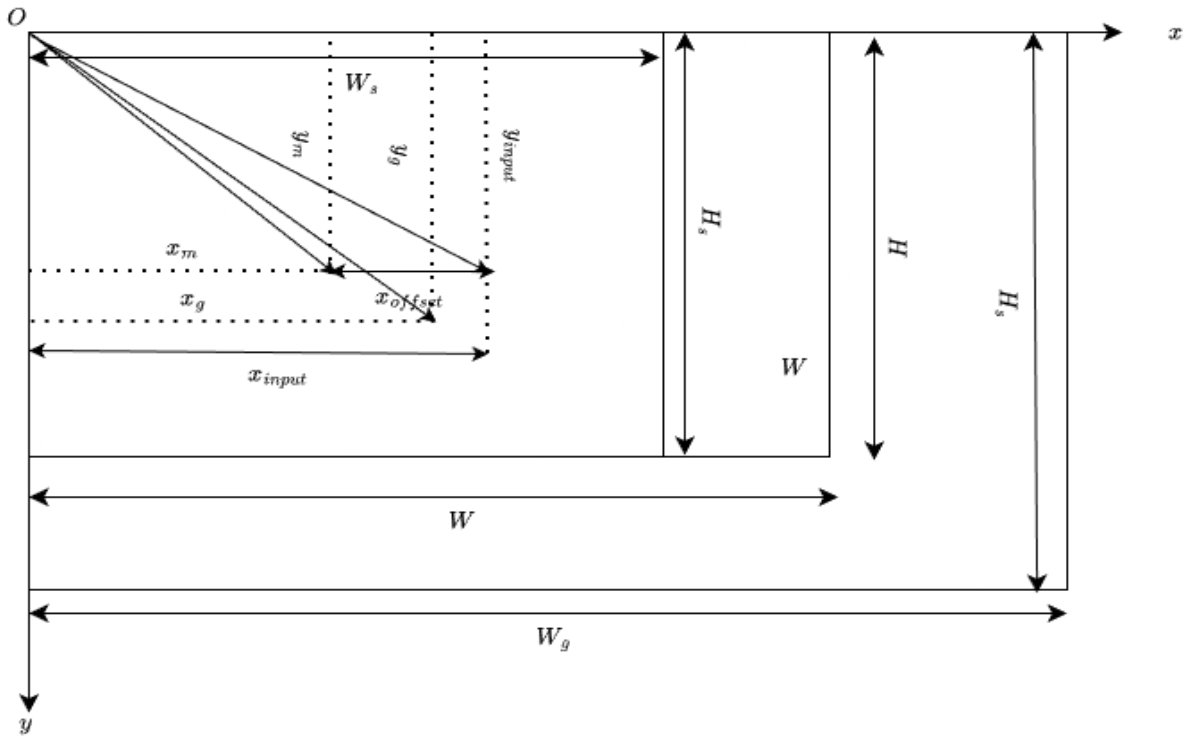
Approach: version 0.1

Coordinate scaling: Target_Setter

The waypoint in global reference frame is given by:

$$x_g = \frac{W_g}{W_s}\left(x_{input} - x_{offset}\right),\ y_g = \frac{H_g}{H_s}\left(y_{input} - y_{offset}\right),\ \theta = \arctan 2\,\frac{y_{input} - y_{odom}}{x_{input} - x_{odom}}$$

This is obtained by applying frame transformation and scaling in Cartesian coordinate system



Approach: version 0.1

Kinematics: ROS2 package

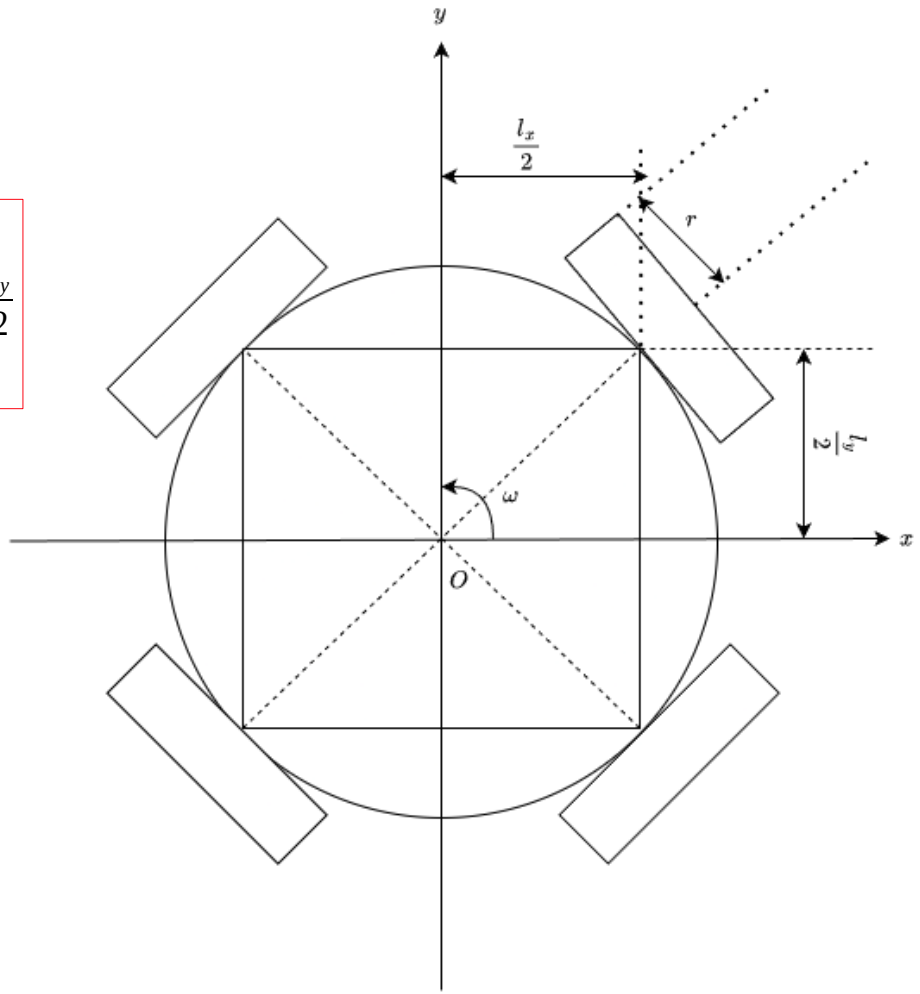
The wheel velocities are given by:

$$\begin{bmatrix} \omega_1[n] \\ \omega_2[n] \\ \omega_3[n] \\ \omega_4[n] \end{bmatrix} = \frac{1}{r} \begin{bmatrix} 1 & -1 & -R \\ 1 & 1 & R \\ 1 & 1 & -R \\ 1 & -1 & R \end{bmatrix} \begin{bmatrix} v_x[n] \\ v_y[n] \\ \omega[n] \end{bmatrix}, R = \frac{l_x}{2} + \frac{l_y}{2}$$

The robot velocities are given by:

$$\begin{bmatrix} v_x[n] \\ v_y[n] \\ \omega[n] \end{bmatrix} = \frac{r}{4} \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 1 & 1 & -1 \\ -\frac{1}{R} & \frac{1}{R} & -\frac{1}{R} & \frac{1}{R} \end{bmatrix} \begin{bmatrix} \omega_1[n] \\ \omega_2[n] \\ \omega_3[n] \\ \omega_4[n] \end{bmatrix}$$

This is obtained by applying kinematics on a rigid body



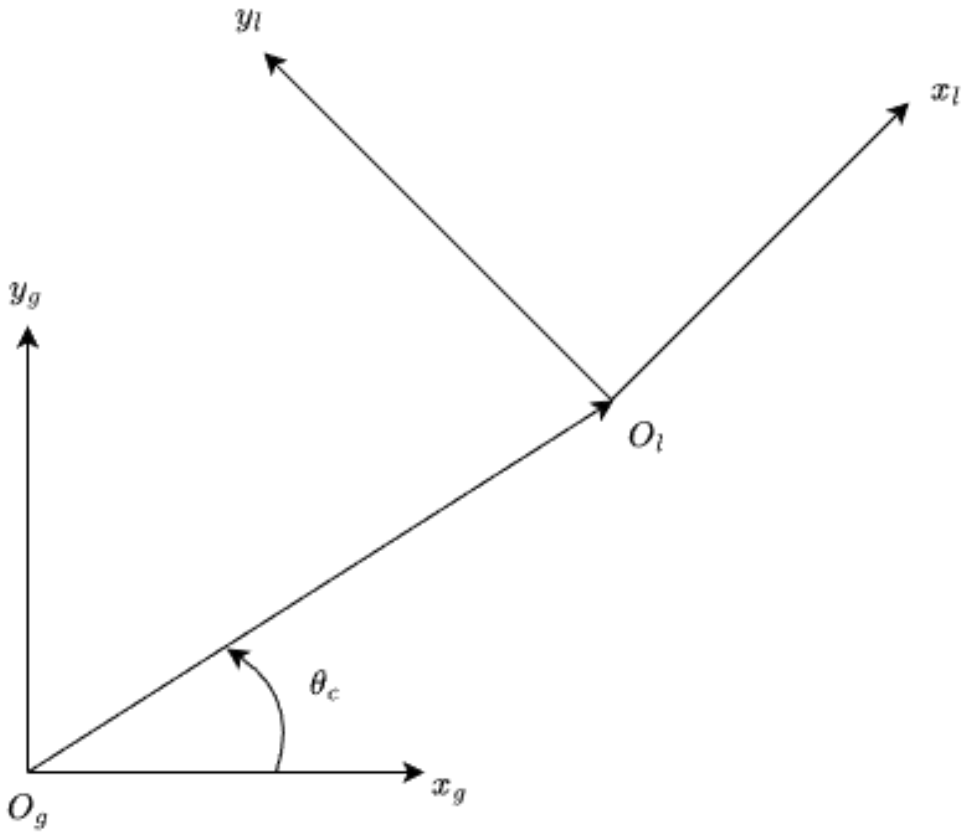
Approach: version 0.1

Frame transformation: ROS2 package

The linear velocities in local frame are given by: $\begin{bmatrix} v_x^l[n] \\ v_y^l[n] \end{bmatrix} = \begin{bmatrix} \cos\theta_c[n] & \sin\theta_c[n] \\ -\sin\theta_c[n] & \cos\theta_c[n] \end{bmatrix} \begin{bmatrix} v_x^g[n] \\ v_y^g[n] \end{bmatrix}$

The linear velocities in global frame are given by: $\begin{bmatrix} v_x^g[n] \\ v_y^g[n] \end{bmatrix} = \begin{bmatrix} \cos\theta_c[n] & -\sin\theta_c[n] \\ \sin\theta_c[n] & \cos\theta_c[n] \end{bmatrix} \begin{bmatrix} v_x^l[n] \\ v_y^l[n] \end{bmatrix}$

This is obtained by applying rotation matrix on linear velocities in global frame and local frame respectively



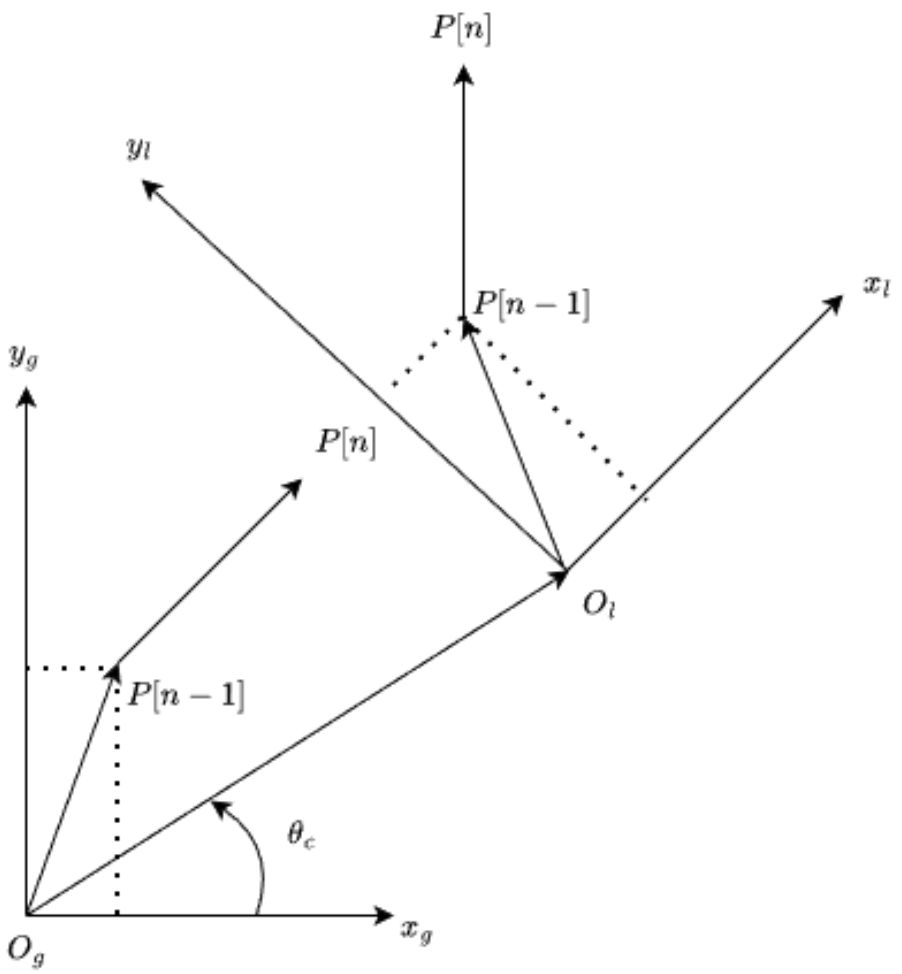
Approach: version 0.1

Odometry: ROS2 package

The robot odometry is defined as:

$$P_{[n+1]} = P[n] + R\begin{bmatrix} v_x[n]T_s \\ v_y[n]T_s \end{bmatrix} \Rightarrow \begin{bmatrix} x_{odom} \\ y_{odom} \end{bmatrix}_{n+1} = \begin{bmatrix} x_{odom} \\ y_{odom} \end{bmatrix}_n + \begin{bmatrix} \cos\theta_c[n] & -\sin\theta_c[n] \\ \sin\theta_c[n] & \cos\theta_c[n] \end{bmatrix}\begin{bmatrix} v_x[n]T_s \\ v_y[n]T_s \end{bmatrix}$$

$$\theta_c[n] = \theta[n] - \theta_{offset}, \theta[n] = \arctan 2\left[\frac{2(\omega[n]x[n] + x[n]y[n])}{1 - 2(y^2[n] + z^2[n])}\right], \theta_{offset} = \theta_{initial}$$



Approach: version 0.1

Control logic: ROS2 package

The velocities in global reference frame is given by:

$$u[n]=\begin{bmatrix} v_x^l[n] \\ v_y^l[n] \\ \omega[n] \end{bmatrix}$$

The global velocities is given by the PD controller:

$$\begin{bmatrix} v_x^g[n] \\ v_y^g[n] \end{bmatrix}=\begin{bmatrix} k_p & 0 \\ 0 & k_p \end{bmatrix}\begin{bmatrix} e_x[n] \\ e_y[n] \end{bmatrix}+\begin{bmatrix} k_d & 0 \\ 0 & k_d \end{bmatrix}\begin{bmatrix} \dot{e}_{x,filtered}[n] \\ \dot{e}_{y,filtered}[n] \end{bmatrix}$$

Where

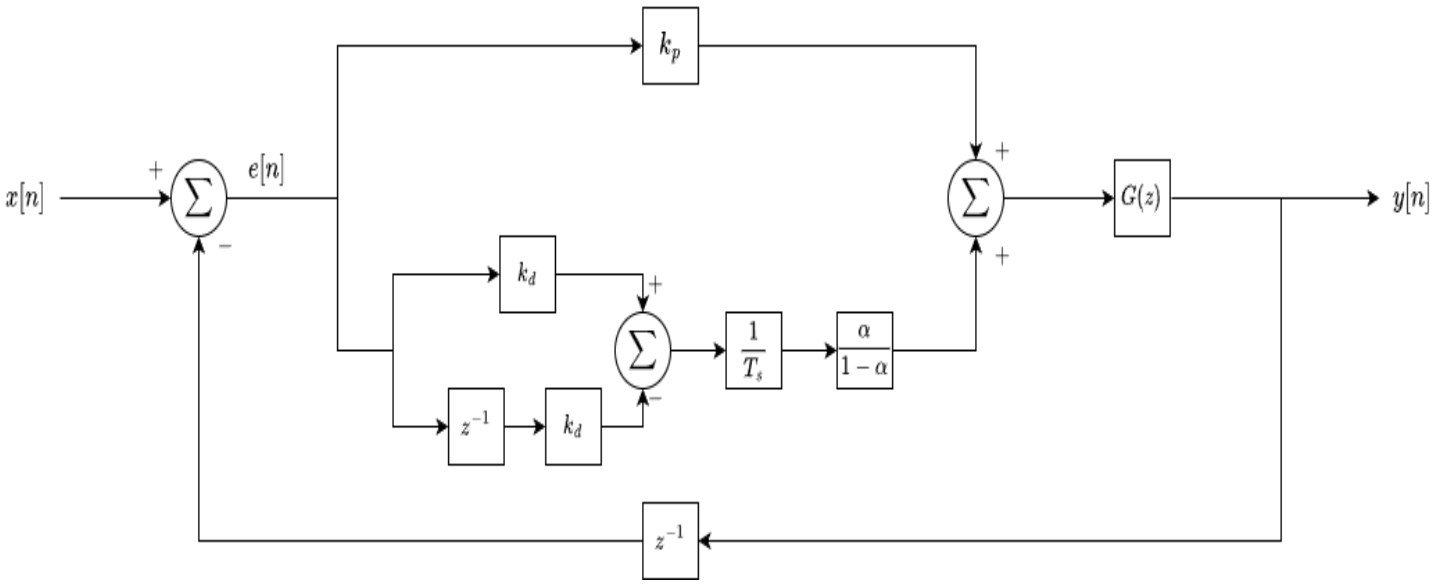
$$\begin{bmatrix} \dot{e}_{x,filtered}[n] \\ \dot{e}_{y,filtered}[n] \end{bmatrix}=\alpha_d\begin{bmatrix} \dot{e}_x[n] \\ \dot{e}_y[n] \end{bmatrix}+(1-\alpha_d)\begin{bmatrix} \dot{e}_x[n-1] \\ \dot{e}_y[n-1] \end{bmatrix}$$

And

$$\begin{bmatrix} e_x[n] \\ e_y[n] \end{bmatrix}=\begin{bmatrix} x_d[n]-x_c[n] \\ y_d[n]-y_c[n] \end{bmatrix},\begin{bmatrix} \dot{e}_x[n] \\ \dot{e}_y[n] \end{bmatrix}=\begin{bmatrix} \dfrac{e_x[n]-e_x[n-1]}{T_s} \\ \dfrac{e_y[n]-e_y[n-1]}{T_s} \end{bmatrix}$$

Then, we calculate local velocities:

$$\begin{bmatrix} v_x^l[n] \\ v_y^l[n] \end{bmatrix}=\begin{bmatrix} \cos\theta_c[n] & \sin\theta_c[n] \\ -\sin\theta_c[n] & \cos\theta_c[n] \end{bmatrix}\begin{bmatrix} v_x^g[n] \\ v_y^g[n] \end{bmatrix}$$



Approach: version 0.1

Control logic: ROS2 package

The angular velocity is given by the PD controller: $\omega[n]=k_p e_\theta[n]$

Where $e_\theta[n]=\theta_d[n]-\theta_c[n]$

The deadzone can be applied: $if\,|v|<\epsilon, v=0$

For safety reason, we limit the robot velocities: $|v|\leq v_{max}, |\omega|\leq \omega_{max}$

Approach: version 0.2

Updates

Version 0.2 is an updates over version 0.1 to improve usability and performance of the system. The update includes:

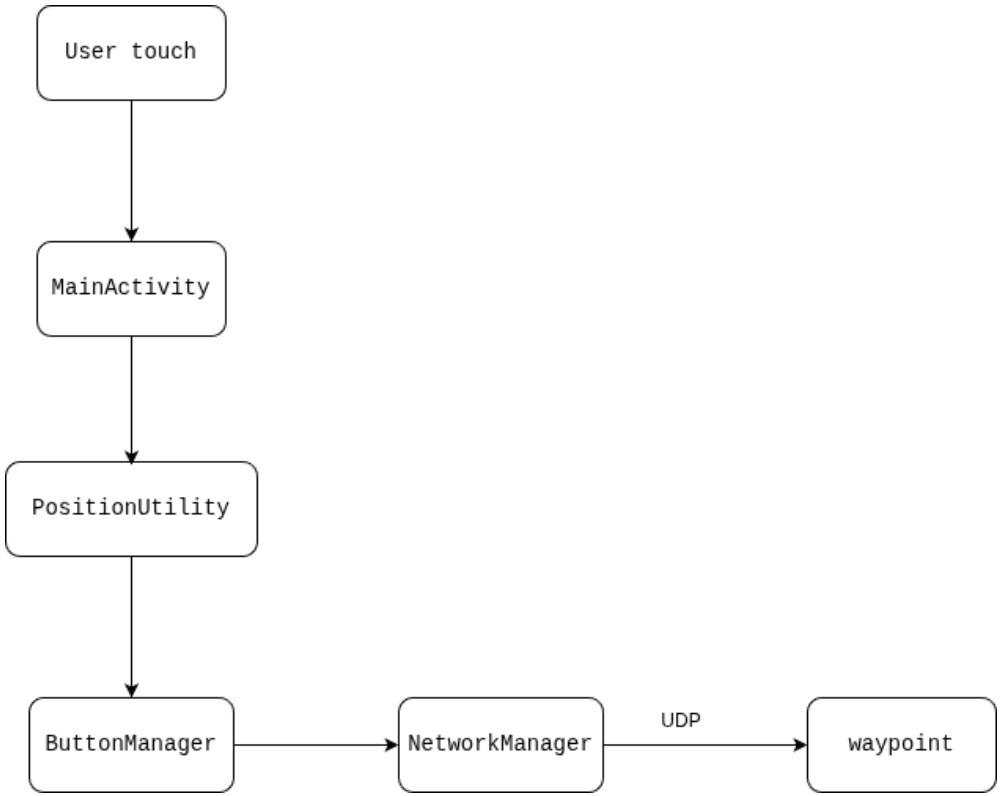
- Target_Setter UI update
- Communication protocol redefinition
- Addition of return and edit functionalities
- Minor system architecture improvement
- Navigation logic addition
- Coordinate scaling update on Target_Setter

Approach: version 0.2

System architecture: Target_Setter update

YawJoystick is removed from Target_Setter app because:

- App doesn’t actually need to calculate yaw because yaw can be inferred from the position inputs
- We assume that the robot is in a planar (2D) environment
- Reduce computational load from the app

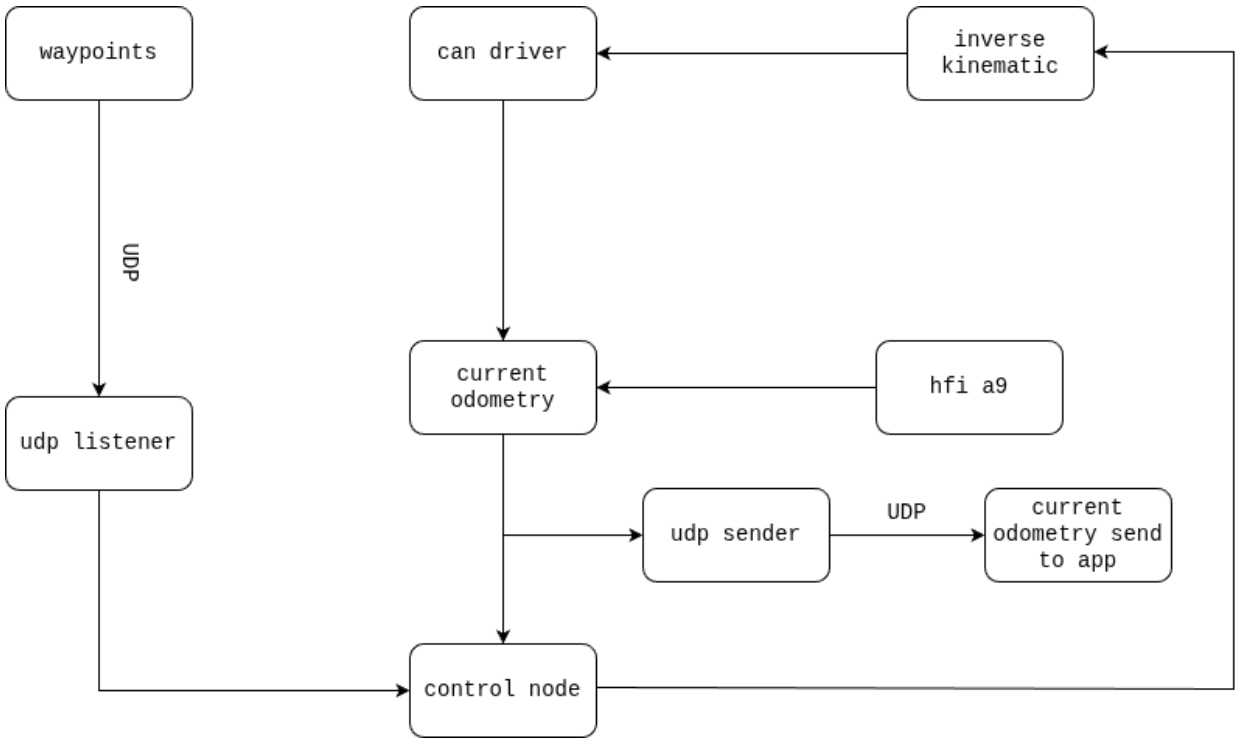


Approach: version 0.2

System architecture: ROS2 package update

target_odometry node is removed from package because:

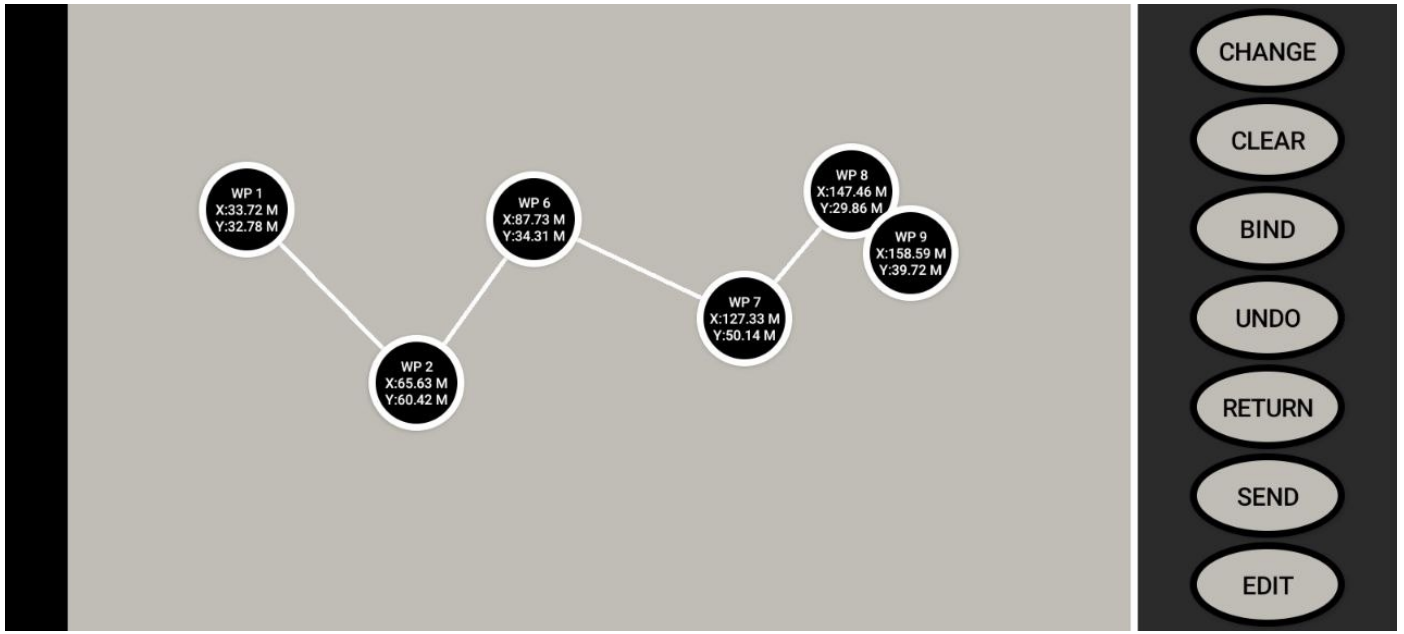
- After redefining communication protocol, target_odometry node is redundant with the udp_listener node
- UDP packets can be processed directly in the udp_listener node



Approach: version 0.2

Android app design update

- Add lines connecting from one waypoint to another for better readability
- Allows users to input gamefield parameters (dimension) for scalability and modularity
- Removed virtual joystick
- Add Edit and Return buttons



Approach: version 0.2

Coordinate scaling

Since we removed yaw input from the UI, the app can no longer provides yaw input. After this change, the waypoints in global frame is provided as:

$$x_g = \frac{W_g}{W_s}\left(x_{input} - x_{offset}\right), y_g = \frac{H_g}{H_s}\left(y_{input} - y_{offset}\right)$$

Yaw correction is handled by the ROS2 package using positions input from app and IMU quaternion readings.

$$\theta_d[n] = \arctan 2\left(\frac{y_d[n] - y_c[n]}{x_d[n] - x_c[n]}\right)$$

Approach: version 0.2

Communication protocol redefinition

- Different packet types for different messages (structure problem)
- Implement binary UDP packet with Little Endian byte order to decrease transmission latency
- Update communication flow by including edit and return functionalities
- udp_listener node update to process the packets properly before publishing decision

Approach: version 0.2

Navigation logic addition

To properly navigate the robot, we add arrival detection.

We can check the distance using this equation $d[n]=\sqrt{(x_d[n]-x_c[n])^2+(y_d[n]-y_c[n])^2}$

Then, we compare the distance with the predefined threshold.

The robot is considered to be arrived at the waypoint if the distance is less than or equal to the threshold.

Approach: version 0.2

Addition of return and edit functionalities

The app send the return and edit packets. Each packet contains flag that signals ROS2 to return or edit the waypoint.

udp_listener node process these packets, then forward it to the control node.

robot_control node implements the logic required for the navigation operation.

Approach: version 0.2

Addition of return and edit functionalities: edit

If the edited waypoint is not active, the robot continues its navigation normally, otherwise, we implement complementary filter to smooth robot navigation, avoiding sudden jumps in trajectory that could cause instability.

This is done using these equations:

$$x_{active}[n]=(1-\alpha)\,x_{active}[n]+\alpha\,x_{edit}[n],\; y_{active}[n]=(1-\alpha)\,y_{active}[n]+\alpha\,y_{edit}[n]$$

$$\theta_{active}[n]=(1-\alpha)\,\theta_{active}[n]+\alpha\,\theta_{edit}[n],\; \theta_{edit}[n]=\arctan 2\left(\frac{x_{edit}[n]-x_{active}[n]}{y_{edit}[n]-y_{active}[n]}\right)$$

Or in matrix form

$$\begin{bmatrix} x_{active}[n] \\ y_{active}[n] \\ \theta_{active}[n] \end{bmatrix}=(1-\alpha)\begin{bmatrix} x_{active}[n] \\ y_{active}[n][n] \\ \theta_{active} \end{bmatrix}+\alpha\begin{bmatrix} x_{edit}[n] \\ y_{edit}[n] \\ \theta_{edit}[n] \end{bmatrix}$$

Approach: version 0.2

Addition of return and edit functionalities: return

To implement return functionality, we need to store waypoints that the robot has gone to. We store it in a stack after the robot has reached the waypoint. It is stored sequentially until the return flag is detected.

If the return flag is detected, we pop the stack. The previous waypoint becomes the active waypoint.

Approach: version 0.3

Updates

Version 0.3 is an update over version 0.2 on navigation logic. The update includes:

- Introduction of state machine
- Yaw correction improvement by adding derivative term with filter

For this version, there is no major update on any module except the robot_control node.

System architecture, Android app, and communication protocol remains the same.

Approach: version 0.3

Introduction of state machine

To enable the robot to navigate properly with the addition of return and edit functionalities, we introduce state machine to avoid undefined behaviors. There are 5 states:

- IDLE: default state
- NAVIGATING: when the robot is moving toward waypoint
- EDIT: when the active waypoint is edited
- PAUSED: when all the motion is stopped either automatically or manually
- RETURN: when the robot is returning to a previous waypoint due to return flag detected

Approach: version 0.3

Yaw correction improvement

Proportional controller alone wasn’t sufficient for yaw correction. This version introduces PD controller for yaw with derivative filter. Angular velocity is given by:

$$\boxed{\omega[n]=k_p e_\theta[n]+k_d \dot{e}_{\theta,filtered}[n]}$$

$$\boxed{e_\theta[n]=\theta_d[n]-\theta_c[n], \dot{e}_\theta[n]=\frac{e_\theta[n]-e_\theta[n-1]}{T_s}}$$

$$\boxed{\dot{e}_{\theta,filtered}[n]=\alpha_{d,\theta}\dot{e}_\theta[n]+(1-\alpha_{d,\theta})\dot{e}_\theta[n-1]}$$

Result & Discussion

Data collection

Although no quantitative data is recorded from the tests, the repeated qualitative observations were recorded and focused on:

- Robot trajectory and waypoint assignment
- Linear and angular velocity profiles
- State transitions
- Behavior near waypoint boundaries
- Target_Setter performance
- Communication system performance

Result & Discussion

Analysis: version 0.1

- Communication system behavior: Target_Setter lagged under high sampling rate
- Control logic behavior:
  - Robot converges toward waypoint
  - Oscillation occurred due to high-frequency noise

Result & Discussion

Analysis: version 0.2

- Control logic performance: system stability was improved due to:
  - Low-pass filtering on D term
  - Deadzone implementation
- Localization drift: localization drift over long distance because:
  - Actuator deadzone causes small cumulative error leads to micro-correction
  - Mismatches between odometry-based yaw and IMU-based yaw
- Undefined state behavior: unintended movement is caused by state transition conflict
- Insufficiency of MVC architectural pattern: race conditions and state conflicts occurred on the app because of more sophisticated communication system

The system code is developed up to version 0.3. However, the evaluation for this version is still pending.

Result & Discussion

System evaluation

- PD controller with derivative filtering and deadzone exhibits critically damped response, achieving smooth waypoint convergence.
- Velocity capping, deadzone compensation, and state machine supervision are necessary to prevent unsafe acceleration and drift.
- Odometry drift, deadzone effects, incremental errors, and distance-based waypoint inference introduce yaw micro-corrections, causing mild oscillations over long distances.
- At a 10 Hz control loop, the system adapts to waypoint modifications dynamically with minimal observable latency.
- The Android MVC implementation becomes prone to race conditions and state conflicts as asynchronous complexity increases.

Result & Discussion

Improvement suggestions

- Verify version 0.3
- Implement a persistent data logging method
- Integrate sensor fusion for localization
- Implement adaptive PD gains
- Enhance state management in Target_Setter app
- Consider using MVVM architectural pattern for Target_Setter app

Conclusion & Next Step(s) / Future work

- PD controller enables stable, smooth motion for omnidirectional robot.
- Velocity limits & state machine ensure safe navigation.
- Minor oscillations from IMU/waypoint errors are well managed.
- Future work: sensor fusion, adaptive gains, logging, and drift reduction.

References

- Singh, S., “MVC Framework: A Modern Web Application Development Approach and Working,” International Research Journal of Engineering and Technology (IRJET), vol. 7, no. 1, pp. 51–55, Jan. 2020. [Online]. Available: https://www.irjet.net/archives/V7/i1/IRJET-V7I111.pdf
- Nandwanya, A., “ConstraintLayout, Demystified – How It Really Works and When You Should Reach for It,” Android Engineers, Sep. 2, 2025. [Online]. Available: https://androidengineers.substack.com/p/constraintlayout-demystified-how. [Accessed: Feb. 12, 2026].
- “MVC Framework – Architecture,” TutorialsPoint, n.d. [Online]. Available: https://www.tutorialspoint.com/mvc_framework/mvc_framework_architecture.htm. [Accessed: Feb. 13, 2026].
- “Model-View-ViewModel (MVVM),” Microsoft Learn, n.d. [Online]. Available: https://learn.microsoft.com/en-us/dotnet/architecture/maui/mvvm. [Accessed: Feb. 13, 2026].
- “Build a Responsive UI with ConstraintLayout,” Android Developers, n.d. [Online]. Available: https://developer.android.com/develop/ui/views/layout/constraint-layout. [Accessed: Feb. 13, 2026].
- “Unidirectional Link vs. Bidirectional Link vs. Loopback Link,” CCNA Classes, May 27, 2021. [Online]. Available: https://ccna-classes.com/ccna-study-resources/unidirectional-link-vs-bidirectional-link-vs-loopback-link/. [Accessed: Feb. 13, 2026].
- Cisco Network Academy, CCNAv7: Introduction to Networks Companion Guide. Indianapolis, IN, USA: Cisco Press, 2020.

References

- Dordal, P. L., “UDP Transport,” in An Introduction to Computer Networks, Desktop ed. 2.0.11, 2012. [Online]. Available: https://intronetworks.cs.luc.edu/current/html/udp.html [Accessed: Feb. 14, 2026].
- Verschelde, J., “Client/Server Interactions,” Programming Tools and File Management 0.7.5 Documentation, Part VI: Network Programming, 2017. [Online]. Available: https://homepages.math.uic.edu/~jan/mcs275/mcs275notes/lec27.html. [Accessed: Feb. 14, 2026].
- Narayana, S., “CS 352: User Datagram Protocol, Lecture 8.1,” Rutgers University, New Brunswick, n.d. [Online]. Available: https://people.cs.rutgers.edu/~sn624/352-S21/lectures/transport-udp-error-detection.pdf. [Accessed: Feb. 14, 2026].
- “Robotic System, Section II: Modeling, Chapter 3: Coordinates and Transformations,” n.d. [Online]. Available: https://motion.cs.illinois.edu/RoboticSystems/CoordinateTransformations.html . [Accessed: Feb. 15, 2026].
- Kutluca, H., “Robot Operating System 2 (ROS 2) Architecture,” Medium, Dec. 6, 2020. [Online]. Available: https://medium.com/software-architecture-foundations/robot-operating-system-2-ros-2-architecture-731ef1867776. [Accessed: Feb. 15, 2026].
- “ROS 2: What Is DDS?” RTI Community, n.d. [Online]. Available: https://community.rti.com/page/ros-2-what-dds. [Accessed: Feb. 15, 2026].

References

- Silberschatz, A., Galvin, P. B., and Gagne, G., Operating System Concepts, 10th ed. Hoboken, NJ, USA: Wiley, 2018.
- Open Robotics, “Nodes,” ROS 2 Documentation: Humble, 2026. [Online]. Available: https://docs.ros.org/en/humble/Concepts/Basic/About-Nodes.html. [Accessed: Feb. 15, 2026].
- Open Robotics, “Topics,” ROS 2 Documentation: Humble, 2026. [Online]. Available: https://docs.ros.org/en/humble/Concepts/Basic/About-Topics.html. [Accessed: Feb. 15, 2026].
- Open Robotics, “Interfaces,” ROS 2 Documentation: Humble, 2026. [Online]. Available: http://docs.ros.org/en/humble/Concepts/Basic/About-Interfaces.html. [Accessed: Feb. 15, 2026].
- Lipschutz, S., and Lipson, M. L., Schaum's Outline of Theory and Problems of Discrete Mathematics, 3rd ed. New York, NY, USA: McGraw-Hill, 2007. doi: 10.1035/0071470387.
- Lee, E. A., and Varaiya, P., Structure and Interpretation of Signals and Systems. Pearson Education, 2003. [Online]. Available: https://ptolemy.berkeley.edu/projects/chess/eecs124/reading/LeeAndVaraiya3_4.pdf. [Accessed: Feb. 15, 2026].
- Åström, K. J., “Control System Design,” 2002. [Online]. Available: https://www.cds.caltech.edu/~murray/courses/cds101/fa02/caltech/astrom-ch6.pdf. [Accessed: Feb. 16, 2026].

References

- Boghossian, A., Brown, J., and Zak, S., “P, I, D, PI, PD, and PID Control,” LibreTexts Engineering, University of Michigan, n.d. [Online]. Available: https://eng.libretexts.org/Bookshelves/Industrial_and_Systems_Engineering/Chemical_Process_Dynamics_and_Controls_(Woolf)/09:_Proportional-Integral-Derivative_(PID)_Control/9.02:_P_I_D_PI_PD_and_PID_control
  . [Accessed: Feb. 16, 2026].
- Oliveira, H. P., et al., “Dynamic Models for Omni-directional Robots with 3 and 4 Wheels,” in Proceedings of the International Conference on Informatics in Control, Automation and Robotics (ICINCO), 2008.
- Phunopas, A., and Inoue, S., “Motion Improvement of Four-Wheeled Omnidirectional Mobile Robots for Indoor Terrain,” Journal of Robotics, Networking and Artificial Life, vol. 4, no. 4, pp. 275–282, Mar. 2018.
- Karnopp, D. C., Margolis, D. L., and Rosenberg, R. C., System Dynamics: Modeling, Simulation, and Control of Mechatronic Systems. Hoboken, NJ, USA: John Wiley & Sons, 2012.
- Lathi, B. P., Principles of Linear Systems and Signals, 2nd ed. Oxford University Press, 2009.
- Simrock, S., “Control Theory,” DESY, Hamburg, Germany, n.d. [Online]. Available: https://cds.cern.ch/record/1100534/files/p73.pdfhttps://cds.cern.ch/record/1100534/files/p73.pdf https://cds.cern.ch/record/1100534/files/p73.pdf. [Accessed: Feb. 16, 2026].